Mastering Programming Fundamentals

From loops to functions, build your coding foundation with hands-on practice and real-world examples.

Looping: Repeat with Precision

```
for (int i = 0; i < 5; i++) {
  print(i);
}
```

This loop runs five times, with `i` starting at 0 and incrementing until it reaches 5. Each iteration prints the current value of `i`.

What You'll See

- Numbers 0 through 4 printed sequentially
- Loop control with start, condition, and increment
- Efficient repetition without writing duplicate code

Functions: Reusable Code Blocks

01

Define the Function

Specify what it returns and what parameters it accepts

02

Write the Logic

Build the code that performs the operation

03

Return Results

Send back computed values to the caller

```
int add(int a, int b) {
  return a + b;
}
```

This function takes two integers and returns their sum.

Arrow Functions: Compact Syntax

Simplified One-Liners

Arrow syntax `=>` creates concise functions when you have a single expression to return.

```
int multiply(int a, int b) => a * b;
```

This is equivalent to the longer function syntax but more compact and readable for simple operations.

When to Use

- Single expression functions
- Callback functions
- Short, clear logic

Lists: Ordered Collections

Creation

```
List fruits =
["Apple", "Mango"];
```

Access

Use indices starting at 0 to retrieve items

Manipulation

Add, remove, or modify elements dynamically

Maps: Key-Value Pairs

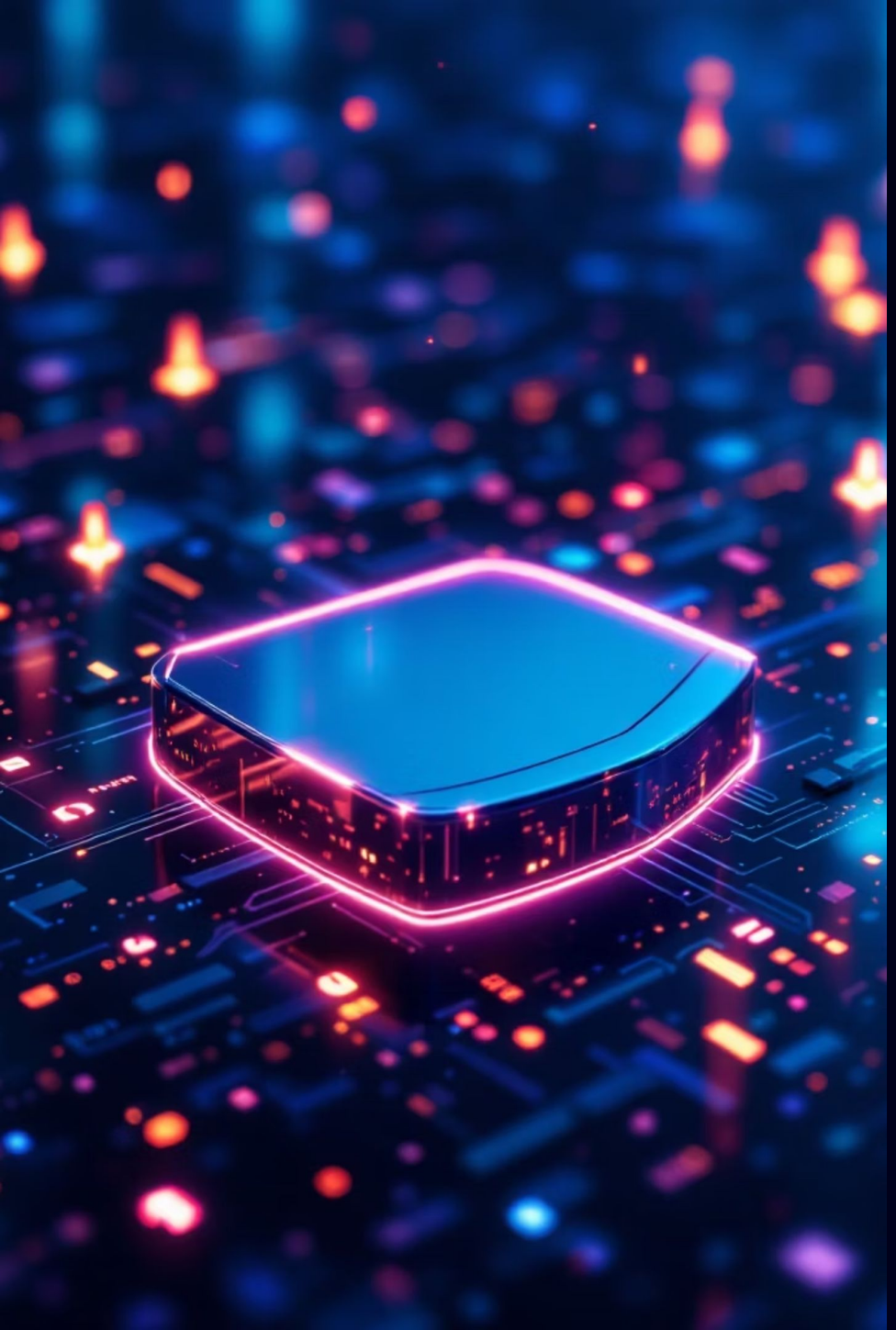
Structure

Maps store data as key-value pairs, allowing you to access values using descriptive keys instead of numeric indices.

```
Map user = {
  "name": "Ali",
  "email": "ali@example.com"
};
```

Benefits

- Meaningful access with string keys
- Flexible data organization
- Efficient lookups by key



Null Safety: Prevent Runtime Errors

The Problem

Null values can cause crashes when you try to access properties or methods on non-existent data.

The Solution

Use `?` after a type to explicitly mark it as nullable: `String? name;`

Safe Access

Check for null before accessing or use null-aware operators to prevent runtime exceptions.

Live Coding Session

Time to put theory into practice! Follow along as we build working code together.

Build a Simple Program

Create a basic structure with variables and output

Modify Variables

Update values and see how changes affect program behavior

Write a Loop

Implement repetition logic to automate tasks

Create a Function

Package reusable logic into a named operation

Interactive Exercises

Print Numbers 1–10

Use a loop to generate a sequence from 1 to 10 inclusive.

Print Even Numbers

Modify your loop to print only even numbers in the range.

Create a Function

Build a function that takes parameters and returns a calculated result.

Final Challenge: Student Info Program

Your Task

1. Store a student's name and marks using variables
2. Write conditional logic to determine grades
3. Print the student's name and corresponding grade (A, B, or C)

Grading Criteria

- A: 80+ marks
- B: 60–79 marks
- C: Below 60 marks

Sample Solution

```
void main() {
 String name = "Ali";
 int marks = 75;

 if (marks >= 80) {
 print("$name got A");
 } else if (marks >= 60) {
 print("$name got B");
 } else {
 print("$name got C");
 }
}
```